MEMORIA TÉCNICA DE PRÁCTICAS

Instalación, Configuración del Entorno Flutter y Primer Proyecto Multiplataforma

Ciclo: 2º DAM (Desarrollo de Aplicaciones Multiplataforma)

Módulo: Programación Móvil / Desarrollo de Interfaces

Entorno del Sistema: Windows 11 / 10 (x64)

Alumno / Autor: Kevin

1. Introducción y Fundamentos del Desarrollo Multiplataforma

En el desarrollo móvil contemporáneo, existen dos filosofías principales: el desarrollo nativo exclusivo y las estrategias multiplataforma (*cross-platform*).

Paradigma	Tecnologías	Mecanismo	Ventajas	Inconvenientes
Web Responsive	HTML5, CSS3, JS	Navegador estándar	100% código único, sin tiendas	Sin acceso profundo a hardware
Híbridas	Ionic, Cordova	WebView del SO	Uso de frameworks web clásicos	Sobrecarga de renderizado en WebView
Puente Nativo	React Native	Bridge JavaScript ↔ Widgets Nativos	Aspecto nativo del sistema	Cuello de botella en el puente JS
Renderizado Directo	**Flutter** (Google)	Código máquina compilado con motor gráfico propio (Skia/Impeller)	**60/120 FPS reales, consistencia visual exacta, Hot Reload**	Binario base ligeramente mayor

¿Por qué Flutter y Dart? Flutter dibuja directamente sobre el lienzo gráfico (Canvas) del sistema operativo sin delegar en componentes nativos del fabricante. El lenguaje **Dart** permite compilación **JIT** (Just-In-Time) durante el desarrollo para habilitar *Stateful Hot Reload*, y compilación **AOT** (Ahead-Of-Time) para generar binarios nativos optimizados para producción.

2. Instalación y Configuración de Android Studio y Android SDK

Para compilar hacia la plataforma Android, Flutter requiere las herramientas del SDK oficial.

- **Ruta de Android Studio:** `D:\aplicaciones`
- **Ruta del SDK de Android:** `C:\Users\kevin\AppData\Local\Android\Sdk`

Instalación de Android SDK Command-line Tools (Componente Crítico)

Por defecto, Android Studio no instala las herramientas de línea de comandos requeridas por Flutter. Para activarlas:

1. Abrir Android Studio → **More Actions** → **SDK Manager**.
2. Navegar a la pestaña **SDK Tools**.
3. Marcar la casilla **Android SDK Command-line Tools (latest)** y pulsar **Apply**.

> **Aceptación de Licencias:** Es indispensable ejecutar en la terminal el comando `flutter doctor --android-licenses` y aceptar cada una de las licencias mostradas con `y`.

3. Instalación y Verificación de Flutter SDK

Se descarga el SDK de Flutter (canal `stable`) y se ubica en un directorio sin caracteres especiales ni requerimientos de privilegios de Administrador (por ejemplo, `C:\src\flutter` o `D:\flutter`). Se añade la subcarpeta `bin` a la variable de entorno `Path`.

Se ejecuta el comando de verificación:

```
flutter doctor -v
```

El asistente valida de forma exhaustiva el SDK de Flutter, la cadena de herramientas de Android (Android toolchain), el navegador Chrome para desarrollo web, la instalación de Android Studio y los dispositivos conectados.

4. Configuración del Editor de Código: VS Code frente a Android Studio

Criterio	Visual Studio Code (Opción Ligera)	Android Studio (Opción Completa)
Consumo de Memoria	~250 - 450 MB (Muy liviano)	~2.5 - 4.0 GB (Elevado)
Arranque	Instantáneo (1-2 segundos)	15-30 segundos indexando
Extensiones / Plugins	Flutter, Dart, Pubspec Assist, Awesome Flutter Snippets	Plugins Flutter y Dart integrados en JetBrains
Uso recomendado	**Desarrollo ágil cotidiano** y portátiles	Edición profunda de módulos nativos Gradle / Kotlin

5. Creación, Estructura y Modificación del Primer Proyecto

Creación del proyecto mediante CLI:

```
flutter create --org com.ies.dam mi_primer_proyecto
```

Estructura del Proyecto

- `lib/main.dart` : Punto de entrada de la aplicación y árbol de Widgets.
- `pubspec.yaml` : Manifiesto de dependencias, tipografías y recursos gráficos (assets).
- `android/` : Contenedor nativo de Android (Gradle, manifiesto y MainActivity).

Modificaciones Realizadas en `lib/main.dart`

1. **Título de cabecera:** `title: 'Exemple d\'aplicació amb Flutter'`.
2. **Texto descriptivo localizado:** `'Has fet tap al botó el següent nombre de vegades:'`.
3. **Modificación de la paleta de color (Material 3):** `colorScheme: ColorScheme.fromSeed(seedColor: Colors.amberAccent)`.
4. **Comprobación de Stateful Hot Reload:** Al pulsar `r` en la consola tras modificar estilos, los cambios se aplican en subsegundos conservando el valor exacto del contador en memoria.

6. Conexión y Ejecución en Emuladores

- **Emulador Android Studio (AVD):** Se crea un dispositivo virtual (ej. Pixel 7 con Android 13/14) desde el *Device Manager*. Se gestiona desde consola mediante `flutter emulators --launch <id>`.
- **Genymotion Desktop:** Emulador alternativo basado en hipervisores (VirtualBox/QEMU) con licencia de uso personal. Al ejecutarse, es detectado automáticamente por ADB y reconocido por `flutter devices`.

7. Conexión y Despliegue en Dispositivo Físico Real

Probar en un terminal físico proporciona el rendimiento real del hardware ARM y acceso inmediato a sensores, cámara y batería.

1. **Ajustes del Smartphone:** Activar *Opciones de desarrollador* pulsando 7 veces consecutivas sobre el *Número de compilación*, y activar **Depuración por USB**.
2. **Conexión USB:** Conectar el cable y autorizar la huella RSA en pantalla. Comprobar con `adb devices` y `flutter devices`. Desplegar con `flutter run -d <device_id>`.
3. **Depuración Wi-Fi (Android 11+):** Vincular mediante escaneo de código QR desde *Pair Devices Using Wi-Fi* en Android Studio.
4. **Espejo y Control con `scrcpy`:** Herramienta open-source que proyecta la pantalla del móvil en el PC con latencia ultra baja (<50ms) y control total con ratón y teclado mediante `scrcpy --always-on-top --stay-awake`.

8. Conclusiones

El entorno de desarrollo configurado ofrece un flujo de trabajo optimizado para el ciclo de 2º DAM. La combinación de la ligereza de Visual Studio Code con el SDK oficial de Android y la potencia del motor gráfico de Flutter permite desarrollar aplicaciones fluidas con una alta velocidad de iteración gracias a Stateful Hot Reload.